

CampusHub 详细设计文档（整合版）

1. 文档说明

本文档用于整合 CampusHub 校园互助服务平台在详细设计阶段完成的全部核心设计成果，作为独立于任务说明文件的正式详细设计文档。文档面向后续编码实现、接口联调、数据库落库、测试设计和课程答辩展示。

本文档整合以下专题成果：

1. 核心类图与类职责设计。
2. 设计模式应用与 SOLID 原则检查结果。
3. RESTful API 规范设计。
4. 数据库 ER 设计、核心建表方案、索引设计与隐私安全设计。
5. 类设计、接口设计与数据库设计之间的一致性映射。

相关附件如下：

1. [类图.png](#)
2. [API规范文档.md](#)
3. [E-R图说明.md](#)
4. [E-R图_互助任务与订单功能_.png](#)
5. [E-R图_社区论坛与系统消息管理_.jpg](#)
6. [数据库索引与隐私安全设计方案.md](#)
7. [SOLID检查清单.md](#)
8. [campus_hub_schema.sql](#)

2. 设计目标与范围

本次详细设计围绕 P0 优先级功能展开，重点覆盖以下业务链路：

1. 用户注册、登录、资料维护与实名认证。
2. 互助任务的发布、浏览、筛选、接单与关闭。
3. 订单的确认、进行中、完成、取消等状态流转。
4. 评价、通知、信用分更新等闭环能力。
5. 论坛帖子、评论、点赞、收藏、系统消息等基础扩展能力。

详细设计遵循以下统一原则：

1. 领域职责划分清晰，避免大类、大接口和强耦合。

- 2. 接口命名、鉴权方式、错误码和返回结构保持统一。
- 3. 数据模型优先满足第三范式，并兼顾高频查询性能。
- 4. 对密码、实名、证件图片等敏感数据采用默认保护策略。

3. 类设计

3.1 核心类图

本系统核心类图见：[类图.png](#)

类图覆盖了用户、任务、订单、评价、通知、仓储抽象、服务接口、状态对象和策略对象，满足“含类、属性、方法、关系”的交付要求。

3.2 核心领域对象与服务

类或接口	关键属性	关键方法	职责说明
UserAccount	id, studentId, passwordHash, currentRole, status	register(), login(), switchRole(), disable()	管理账号身份、登录状态和账号可用性
UserProfile	userId, nickname, college, contact, avatarUrl	updateProfile()	管理用户公开资料
CreditProfile	userId, creditScore, completedOrderCount, violationCount	refreshScore()	管理信用分画像与信用统计
ServiceRequest	id, requesterId, title, description, category, location, expectedTime, status	publish(), assignProvider(), close()	管理互助任务生命周期

类或接口	关键属性	关键方法	职责说明
ServiceOrder	id, requestId, requesterId, providerId, status, createdAt, finishedAt	accept(), confirm(), start(), complete(), cancel()	管理 接单 后的 订单 流转
Review	id, orderId, reviewerId, revieweeId, rating, comment	submit()	管理 订单 完成 后的 评价
Notification	id, receiverId, type, content, isRead	markAsRead()	管理 站内 通知
RequestService	无状态应用服务接口	publishRequest(), browseRequests(), getRequestDetail(), closeRequest()	对外 提供 任务 发布 与查 询能 力
OrderService	无状态应用服务接口	acceptRequest(), confirmOrder(), startOrder(), completeOrder(), cancelOrder()	对外 提供 订单 流转 能力
ReviewService	无状态应用服务接口	submitReview(), getUserReviews()	对外 提供 评价 能力
UserRepository	仓储抽象	save(), findById(), findByStudentId()	屏蔽 用户 数据 持久 化细 节

类或接口	关键属性	关键方法	职责说明
RequestRepository	仓储抽象	save(), findById(), findOpenRequests()	屏蔽 任务 数据 持久 化细 节
OrderRepository	仓储抽象	save(), findById(), findByRequestId()	屏蔽 订单 数据 持久 化细 节
ReviewRepository	仓储抽象	save(), findById(), findByRevieweeId()	屏蔽 评价 数据 持久 化细 节
NotificationGateway	网关抽象	send(receiverId, type, content)	屏蔽 具体 通知 发送 实现
OrderState	状态接口	accept(order), confirm(order), start(order), complete(order), cancel(order)	抽象 订单 状态 差异
PendingState / AcceptedState / InProgressState / CompletedState / CancelledState	具体状态对象	按状态实现不同流转规则	封装 状态 行为

类或接口	关键属性	关键方法	职责说明
MatchingStrategy	策略接口	matchRequests(userId, requestList)	抽象任务匹配规则
BasicMatchingStrategy	具体策略实现	matchRequests(...)	当前默认匹配规则
CreditPolicy	策略接口	calculate(profile, reviews)	抽象信用分计算规则
DefaultCreditPolicy	具体策略实现	calculate(...)	当前默认信用策略

3.3 类之间的核心关系

1. UserAccount 与 UserProfile、CreditProfile 是 1:1 拥有关系，分别表示账号主体、个人资料和信用画像。
2. UserAccount 与 ServiceRequest 是 1:N 发布关系，一个用户可发布多个任务。
3. ServiceRequest 与 ServiceOrder 是 1:0...1 生成关系，一个任务在首版业务中最多对应一个有效订单。
4. UserAccount 与 ServiceOrder 是 1:N 参与关系，用户既可以是发布者，也可以是接单者。
5. ServiceOrder 与 Review 是 1:0...2 关系，一个订单完成后最多产生双方两条评价。
6. UserAccount 与 Notification 是 1:N 接收关系。
7. ServiceOrder 组合 OrderState，通过当前状态对象控制订单流转行为。
8. 应用服务依赖 Repository 和 Gateway 抽象，而不依赖具体数据库或消息实现。

4. 设计模式与 SOLID 设计审查

4.1 设计模式应用

4.1.1 状态模式

应用位置：ServiceOrder + OrderState

原因：订单存在 Pending、Accepted、InProgress、Completed、Cancelled 等多种状态，不同状态允许的操作不同。使用状态模式可以把流转规则从大量条件分支中拆出，避免 OrderService 成为流程分支中心。

若不使用：

1. 订单状态相关代码会集中在服务层并不断膨胀。
2. 新增状态时需要频繁修改旧逻辑，容易引入回归问题。
3. 非法状态流转难以统一约束。

4.1.2 策略模式

应用位置：MatchingStrategy、CreditPolicy

原因：任务匹配规则和信用分规则都可能随着业务发展进行调整。使用策略模式后，规则扩展可以通过新增实现类完成，而不需要修改核心业务流程。

若不使用：

1. 服务层会硬编码多套业务规则。
2. 新增规则时必须改动既有核心类。
3. 单元测试难以隔离不同规则分支。

4.1.3 仓储模式与网关抽象

应用位置：UserRepository、RequestRepository、OrderRepository、ReviewRepository、NotificationGateway

原因：业务层依赖抽象接口而不是依赖 MySQL、WebSocket 等具体实现，便于后续替换基础设施或引入测试替身。

若不使用：

1. 高层服务会直接耦合数据库实现类和通知实现类。
2. 技术栈变化会直接冲击业务逻辑层。
3. 单元测试与集成测试的隔离成本更高。

4.2 AI 设计缺陷注入实验结论

根据 [SOLID检查清单.md](#)，AI 初版类设计共发现 5 类典型问题，分别对应 SOLID 五项原则：

1. 单一职责原则：CampusHubManager 同时承担用户、任务、订单、评价、通知和统计职责。
2. 开闭原则：RequestService 和 OrderService 初版依赖大量条件分支处理分类和状态。
3. 里氏替换原则：PendingOrder、CompletedOrder 等继承 ServiceOrder，但行为不一致。
4. 接口隔离原则：统一 CampusHubService 接口过于庞大，调用方被迫依赖不必要的方法。
5. 依赖倒转原则：高层服务直接依赖 MySqlOrderRepository、WebSocketNotificationSender 等具体实现。

修正后的设计采取了以下措施：

1. 将大类拆分为多个领域对象和应用服务接口。
2. 将订单状态差异迁移到 OrderState 体系中。
3. 将匹配规则和信用规则抽象为策略接口。
4. 将统一大接口拆分为 RequestService、OrderService、ReviewService、NotificationGateway 等小接口。
5. 让业务层统一依赖 Repository 和 Gateway 抽象。

结论：AI 适合作为详细设计的初稿生成工具，但不能直接作为最终设计结果落地。经过 SOLID 审查后，当前方案在可维护性、可扩展性和可测试性方面明显更稳定。

5. API 详细设计

完整接口规范见：[API规范文档.md](#)

5.1 统一约定

1. 服务基础前缀统一为 /api。
2. 请求与响应数据格式统一为 application/json。
3. 时间字段统一采用 ISO 8601 字符串。
4. 业务接口默认要求 JWT 鉴权，注册和登录接口允许匿名访问。
5. 所有接口统一返回 ApiResponse<T> 结构，分页列表统一返回 PageResponse<T> 结构。

成功响应示例：

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

失败响应示例：

```
{
  "code": 422,
  "message": "business validation failed",
  "data": null
}
```

5.2 通用错误码

错误码	含义	典型场景
200	成功	请求处理成功
400	请求格式错误	缺少必填字段、字段类型错误、分页参数非法
401	未认证或 Token 失效	未登录访问业务接口、Token 过期
403	无权限	越权操作或普通用户访问管理员接口
404	资源不存在	用户、任务、订单、帖子不存在
409	资源冲突	重复注册、重复接单、重复评价
422	业务校验失败	评分超范围、状态不允许变更
500	服务器内部错误	未预期异常

5.3 核心接口摘要

模块	接口	方法	路径	鉴权	说明
用户与认证	用户注册	POST	/api/user/register	否	创建账号并校验用户基础信息
用户与认证	用户登录	POST	/api/user/login	否	返回 userId、role、token

模块	接口	方法	路径	鉴权	说明
用户与认证	获取个人资料	GET	/api/user/profile	是	获取当前用户完整资料
用户与认证	更新个人资料	PUT	/api/user/profile	是	更新昵称、头像、联系方式等资料
用户与认证	提交实名认证	POST	/api/user/verification/submit	是	提交真实姓名、学号、学院和证件材料
任务	发布任务	POST	/api/tasks	是	发布互助任务
任务	获取任务详情	GET	/api/tasks/{taskId}	否	查看任务详情
任务	获取任务列表	GET	/api/tasks	否	支持分类、关键词、排序、分页筛选

模块	接口	方法	路径	鉴权	说明
任务	更新任务	PUT	/api/tasks/{taskId}	是	发布者修改未关闭任务
任务	删除任务	DELETE	/api/tasks/{taskId}	是	发布者删除任务
订单	抢单	POST	/api/orders/grab	是	根据 taskId 创建订单并占用任务
订单	获取订单详情	GET	/api/orders/{orderId}	是	查看订单状态、任务信息与双方信息
订单	确认接单	POST	/api/orders/{orderId}/confirm	是	由发布者确认接单
订单	完成订单	POST	/api/orders/{orderId}/complete	是	订单完成并允许评价
订单	取消订单	POST	/api/orders/{orderId}/cancel	是	在允许阶段取消订单
评价	提交评价	POST	/api/reviews	是	订单完成后提交评分和评论

模块	接口	方法	路径	鉴权	说明
评价	查看用户评价	GET	/api/reviews/user/{userId}	否	获取某用户历史评价

5.4 API 设计审查结果

在 AI 辅助生成接口初稿后，团队对接口文档进行了统一修正，重点体现在以下方面：

- 1. 统一接口命名与资源路径风格，避免同类资源使用不一致命名。
- 2. 为核心接口补充统一错误码和失败响应结构。
- 3. 明确参数类型、是否必填、分页约束和业务状态约束。
- 4. 明确 JWT 认证边界，以及管理员接口的角色限制。
- 5. 对公开用户信息接口做字段裁剪，避免直接返回手机号、邮箱、学号等敏感信息。

6. 数据库详细设计

数据库相关专题成果见：

- 1. [E-R图说明.md](#)
- 2. [数据库索引与隐私安全设计方案.md](#)
- 3. [campus_hub_schema.sql](#)

6.1 ER 设计概览

ER 图由两部分组成：

- 1. [E-R图_互助任务与订单功能.png](#)
- 2. [E-R图_社区论坛与系统消息管理.jpg](#)

核心实体包括：

- 1. 用户 u_user
- 2. 用户认证 t_user_verification
- 3. 互助任务 t_task
- 4. 任务收藏 t_task_favorite
- 5. 订单 t_order
- 6. 评价 t_review
- 7. 站内消息 t_message

- 8. 订单聊天消息 t_chat_message
- 9. 系统公告 t_notice
- 10. 论坛帖子 t_post
- 11. 论坛评论 t_comment
- 12. 帖子点赞 t_post_like
- 13. 帖子收藏 t_post_favorite
- 14. 举报 t_report

6.2 核心实体关系

关系	基数	说明
用户 -> 互助任务	1:N	一个用户可以发布多个任务
互助任务 -> 订单	1:0...1	一个任务最多生成一个有效订单
用户 -> 订单	1:N	用户可作为发布者或接单者参与订单
订单 -> 评价	1:0...2	一个订单最多产生双方两条评价
用户 <-> 互助任务	M:N	通过 t_task_favorite 实现收藏关系
用户 -> 站内消息	1:N	一个用户可以接收多条消息
订单 -> 聊天消息	1:N	一个订单下可以产生多条聊天记录
用户 -> 论坛帖子	1:N	一个用户可以发布多个帖子
论坛帖子 -> 评论	1:N	一个帖子包含多条评论
用户 <-> 论坛帖子	M:N	通过点赞和收藏中间表建立互动关系

6.3 核心表设计摘要

表名	关键字段	设计要点
u_user	id, student_id, password, role, credit_score, status	学号唯一，密码仅保存哈希，支持逻辑删除
t_user_verification	user_id, reviewer_id, real_name, student_card_image, status	实名信息独立存储，便于权限控制和加密保护

表名	关键字段	设计要点
t_task	publisher_id, title, category, location, reward, status	支撑任务大厅的筛选、排序和发布管理
t_order	task_id, poster_id, helper_id, status, version	支撑订单流转与并发控制
t_review	order_id, reviewer_id, target_user_id, rating, content	支撑双向评价与信用反馈
t_message	receiver_id, type, is_read	支撑站内通知和未读统计
t_chat_message	order_id, sender_id, receiver_id, create_time	支撑订单维度聊天记录查询
t_post / t_comment	author_id, category, counters, reply_to_comment_id	支撑论坛互动与评论回复

6.4 索引设计

索引设计基于高频业务读写场景，而不是简单地“有外键就建索引”。核心方案如下：

表	索引	设计目的
u_user	唯一索引 student_id	防止并发重复注册同一学号
t_task	idx_status_category_time(status, category, create_time DESC)	支撑任务大厅按状态、分类、时间倒序查询
t_order	idx_poster_status(poster_id, status)	支撑“我发布的订单”分页查询
t_order	idx_helper_status(helper_id, status)	支撑“我接单的订单”分页查询
t_order	idx_task_id(task_id)	支撑任务与订单的一对一查询
t_message	idx_receiver_read(receiver_id, is_read)	支撑未读消息数量统计

表	索引	设计目的
t_chat_message	idx_order_time(order_id, create_time DESC)	支撑订单聊天历史查询
t_post	idx_priority_time(is_top, is_recommended, create_time DESC)	支撑置顶和推荐帖子流
t_post	idx_category_time(category, create_time DESC)	支撑板块帖子分页查询
t_task_favorite	uk_user_task(user_id, task_id)	防止重复收藏任务
t_post_like	uk_user_post(user_id, post_id)	防止重复点赞
t_post_favorite	uk_user_post_fav(user_id, post_id)	防止重复收藏帖子

6.5 隐私与安全设计

6.5.1 密码安全

1. 密码禁止明文存储。
2. 禁止使用 MD5 或静态盐哈希方案。
3. 必须使用 BCrypt 或 Argon2 进行动态加盐存储。

6.5.2 个人身份信息保护

1. 真实姓名、手机号、身份证号等 PII 采用 AES-256-GCM 或 SM4 进行对称加密入库。
2. 加密密钥通过 KMS 或独立配置注入托管，不与数据库部署在同一位置。
3. 学生证等证件图片仅保存对象存储 ObjectKey，不保存公开可遍历 URL。
4. 审核场景使用短时效预签名 URL，降低链接泄露风险。

6.5.3 接口脱敏

1. 后端不直接返回完整用户实体给前端。
2. 对外统一使用 DTO 或 VO，仅暴露业务必需字段。
3. 若需要展示姓名或手机号，由后端先做脱敏处理后再返回前端。

6.6 数据库设计审查结论

1. 数据模型整体满足第三范式，核心实体边界清晰。
2. 高并发列表查询和重复写入风险已通过组合索引和联合唯一约束处理。
3. 订单表引入 version 字段，为后续乐观锁并发控制提供基础。

4. 论坛、举报、消息等扩展表为后续演进留出了空间。

7. 设计映射与一致性说明

为了保证类设计、接口设计和数据库设计的一致性，核心映射如下：

业务能力	领域类或服务	API 接口	数据表
用户注册与登录	UserAccount, UserRepository	/api/user/register, /api/user/login	u_user
用户资料维护	UserProfile	/api/user/profile	u_user
实名认证	UserAccount, NotificationGateway	/api/user/verification/submit	t_user_verification, t_message
任务发布与浏览	ServiceRequest, RequestService	/api/tasks, /api/tasks/{taskId}	t_task, t_task_favorite

业务能力	领域类或服务	API 接口	数据表
抢单与订单流转	ServiceOrder, OrderService, OrderState	/api/orders/grab, /api/orders/{orderId}/*	t_order, t_message, t_chat_message
评价与信用更新	Review, ReviewService, CreditProfile, CreditPolicy	/api/reviews	t_review, u_user
通知提醒	Notification, NotificationGateway	由相关业务接口触发	t_message
论坛互动	可扩展帖子与评论服务	帖子、评论、点赞、收藏接口	t_post, t_comment, t_post_like, t_post_favorite

该映射说明以下三点：

1. 每项核心业务能力都能同时落到对象设计、接口设计和数据设计三个层面。
2. 任务、订单、评价链路在类图、API 文档和数据库模型之间保持一致。
3. 敏感数据保护同时体现于对象边界、接口返回和数据库存储策略之中。

8. 总结

本整合版详细设计文档已将类图设计、设计模式与 SOLID 审查、API 规范设计、数据库设计、索引方案和隐私安全设计统一收敛为一份独立文档，能够直接作为本阶段提交材料使用。

整合后的方案具备以下特点：

1. 设计职责边界清晰，避免了 AI 初稿中常见的大类、大接口与强耦合问题。

2. API 规范覆盖核心业务流程，接口风格、错误码与认证策略统一。
3. 数据库设计同时兼顾规范化、查询性能与敏感数据保护。
4. 文档与现有类图、ER 图、SQL 和专题说明文件保持一致，可支撑实现、测试与答辩。